

# DD2459 Software Reliability

## Lab - 3

Md Sakibul Islam

[msis3@kth.se](mailto:msis3@kth.se)

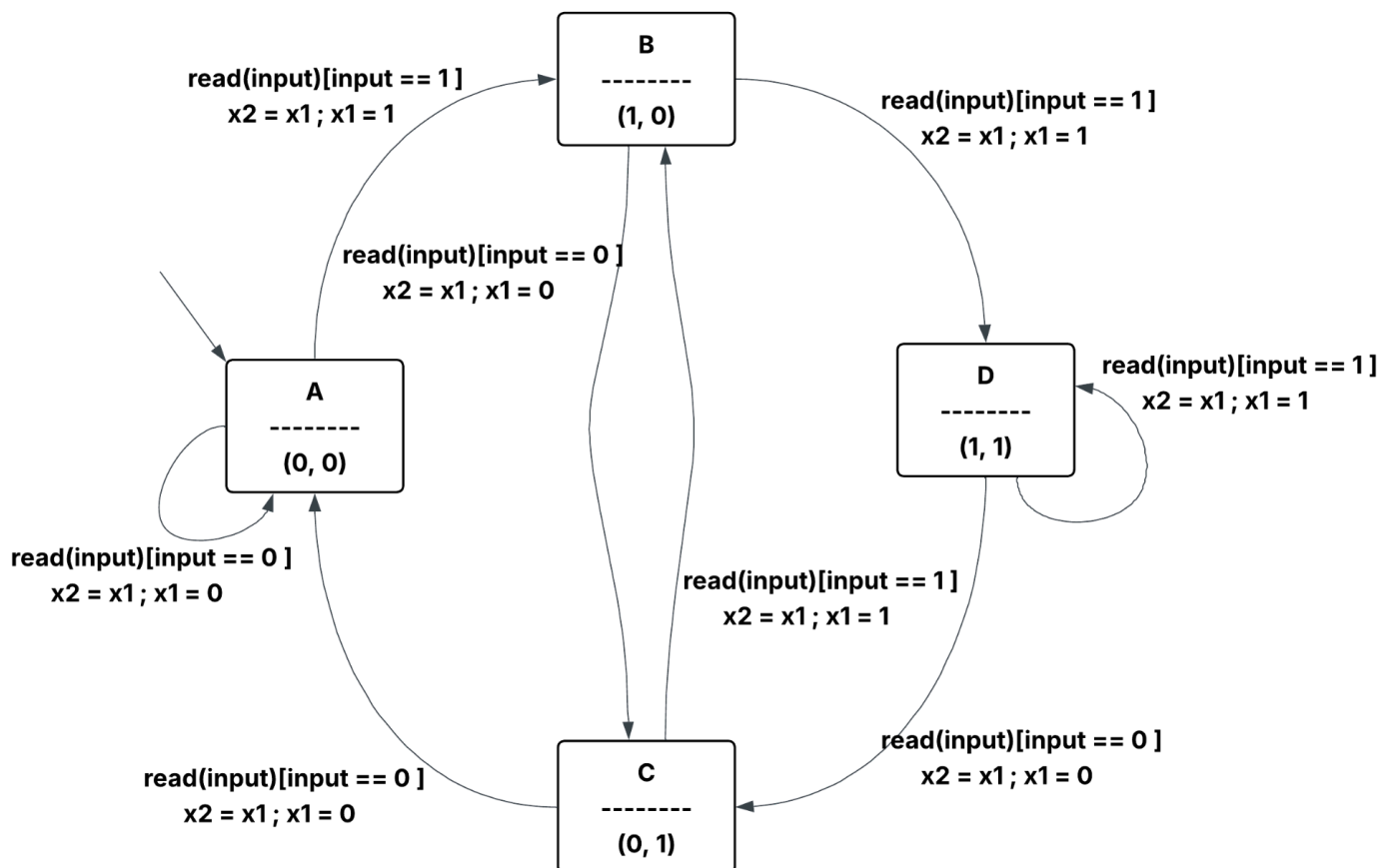
Md Ahsanul Karim

[makari@kth.se](mailto:makari@kth.se)

White-box Testing

Mar 7, 2026

**Exercise - 1:** Draw a model of the 2-bit shift register using in UML



**Exercise - 2 :** Completing the file “bitshift.smv” according to statechart.

**Answer:** According to the statechart, we can find the the next state will be

**next(Bit2):= Bit1**

**next(Bit1):= Input**

The output after completing the code in the NuSMV:

```
-- no counterexample found with bound 0
-- no counterexample found with bound 1
-- no counterexample found with bound 2
-- no counterexample found with bound 3
-- no counterexample found with bound 4
-- no counterexample found with bound 5
-- no counterexample found with bound 6
-- no counterexample found with bound 7
-- no counterexample found with bound 8
-- no counterexample found with bound 9
-- no counterexample found with bound 10
-- no counterexample found with bound 0
-- specification G (Bit1 <-> X Bit1) is false
-- as demonstrated by the following execution sequence
Trace Description: BMC Counterexample
Trace Type: Counterexample
-> State: 1.1 <-
  Bit1 = FALSE
  Bit2 = FALSE
  state = s1
-> Input: 1.2 <-
  input = TRUE
-> State: 1.2 <-
  Bit1 = TRUE
  state = s2
```

From the result our findings are:

a) For Bit2 -> X Bit1

For this test, we can't find a counterexample of the first trap property, which agrees with the definition.

b) For Bit1 -> X Bit1

The test generates this output as counterexample:

| Test | State    | Input | Output    | Correctness of the test |
|------|----------|-------|-----------|-------------------------|
| 1    | S1 (0,0) | 1     | S2 (1, 0) | False                   |

For the trap property, the output Bit1=1 doesn't equal the Bit1 in previous, so the second definition of the trap property is wrong.

### Exercise - 3 : 100% Node Coverage for Car Controller

#### Answer:

a) From the report from NuSMV tool:

```
-- specification G !(state = stop)  is false
-- as demonstrated by the following execution sequence
Trace Description: BMC Counterexample
Trace Type: Counterexample
-> State: 1.1 <-
    state = stop
-- no counterexample found with bound 0
-- specification G ((state = stop & accelerate) -> X !(state = slow))  is
false
-- as demonstrated by the following execution sequence
Trace Description: BMC Counterexample
Trace Type: Counterexample
-> State: 2.1 <-
    state = stop
-> Input: 2.2 <-
    accelerate = TRUE
    brake = FALSE
```

```
-> State: 2.2 <-
state = slow
```

From the output we can find this counterexample :

| Test | Trap Property     | CounterExample |
|------|-------------------|----------------|
| 1    | G !(state = stop) | state = stop   |

b) TR: state = stop

|         | T = 0 | T = 1 | T = 2 | T = 3 |
|---------|-------|-------|-------|-------|
| Inputs  | /     |       |       |       |
| Outputs | stop  |       |       |       |

c) For two additional examples we get this output below :

```
-- specification G !(state = stop) is false
-- as demonstrated by the following execution sequence
Trace Description: BMC Counterexample
Trace Type: Counterexample
-> State: 1.1 <-
state = stop
-- no counterexample found with bound 0
-- specification G !(state = slow) is false
-- as demonstrated by the following execution sequence
Trace Description: BMC Counterexample
Trace Type: Counterexample
-> State: 2.1 <-
state = stop
-> Input: 2.2 <-
accelerate = TRUE
brake = FALSE
-> State: 2.2 <-
state = slow
```

```
-- no counterexample found with bound 0
-- no counterexample found with bound 1
-- specification G !(state = fast) is false
-- as demonstrated by the following execution sequence
```

Trace Description: BMC Counterexample

Trace Type: Counterexample

```
-> State: 3.1 <-
    state = stop
-> Input: 3.2 <-
    accelerate = TRUE
    brake = FALSE
-> State: 3.2 <-
    state = slow
-> Input: 3.3 <-
-> State: 3.3 <-
    state = fast
```

For TR: state = slow

|         | t=0                    | t=1  | t=2 | t=3 |
|---------|------------------------|------|-----|-----|
| Inputs  | Accelerate &<br>!Brake | \    |     |     |
| Outputs | stop                   | slow |     |     |

For TR: state = fast

|         | t=0                    | t=1                    | t=2  | t=3 |
|---------|------------------------|------------------------|------|-----|
| Inputs  | Accelerate &<br>!Brake | Accelerate &<br>!Brake | \    |     |
| Outputs | stop                   | slow                   | fast |     |

## Exercise - 4 : 100% Edge Coverage for Car Controller

a) The output :

```
-- no counterexample found with bound 0
-- specification G ((state = stop & accelerate) -> X !(state = slow)) is false
-- as demonstrated by the following execution sequence
```

Trace Description: BMC Counterexample

Trace Type: Counterexample

```
-> State: 1.1 <-
    state = stop
-> Input: 1.2 <-
    accelerate = TRUE
    brake = FALSE
-> State: 1.2 <-
    state = slow
```

| Test | Trap Property                                        | counterExample                                               |
|------|------------------------------------------------------|--------------------------------------------------------------|
| 1    | (state = stop & accelerate)<br>- > X !(state = slow) | (state=stop & accelerate & !brake -><br>X( !(state=slow) ) ) |

b)Counterexample

|         | t=0                    | t=1  | t=2 | t=3 |
|---------|------------------------|------|-----|-----|
| Inputs  | Accelerate &<br>!Brake | \    |     |     |
| Outputs | stop                   | slow |     |     |

c) Remaining 5 EC trap properties :

1. LTLSPEC

G( state=slow & (accelerate & !brake) -> X( !(state=fast) ) )

2. LTLSPEC

G( state=slow & (!accelerate & !brake) -> X( !(state=stop) ) )

### 3. LTLSPEC

$G(\text{state}=\text{slow} \ \& \ \text{brake} \rightarrow X(\neg(\text{state}=\text{stop}) \ ))$

### 4. LTLSPEC

$G(\text{state}=\text{fast} \ \& \ (\neg\text{accelerate} \ \& \ \neg\text{brake}) \rightarrow X(\neg(\text{state}=\text{slow}) \ ))$

### 5. LTLSPEC

$G(\text{state}=\text{fast} \ \& \ \text{brake} \rightarrow X(\neg(\text{state}=\text{stop}) \ ))$

## The Output :

```
-- no counterexample found with bound 0
-- specification G ((state = stop & accelerate) -> X !(state = slow)) is false
-- as demonstrated by the following execution sequence
Trace Description: BMC Counterexample
Trace Type: Counterexample
-> State: 1.1 <-
    state = stop
-> Input: 1.2 <-
    accelerate = TRUE
    brake = FALSE
-> State: 1.2 <-
    state = slow
-- no counterexample found with bound 0
-- no counterexample found with bound 1
-- specification G ((state = slow & (accelerate & !brake)) -> X !(state = fast)) is
false
-- as demonstrated by the following execution sequence
Trace Description: BMC Counterexample
Trace Type: Counterexample
-> State: 2.1 <-
    state = stop
-> Input: 2.2 <-
    accelerate = TRUE
    brake = FALSE
```

```

-> State: 2.2 <-
    state = slow
-> Input: 2.3 <-
-> State: 2.3 <-
    state = fast
-- no counterexample found with bound 0
-- no counterexample found with bound 1
-- specification G ((state = slow & (!accelerate & !brake)) -> X !(state = stop))
is false
-- as demonstrated by the following execution sequence
Trace Description: BMC Counterexample
Trace Type: Counterexample
-- Loop starts here
-> State: 3.1 <-
    state = stop
-> Input: 3.2 <-
    accelerate = TRUE
    brake = FALSE
-> State: 3.2 <-
    state = slow
-> Input: 3.3 <-
    accelerate = FALSE
-> State: 3.3 <-
    state = stop
-- no counterexample found with bound 0
-- no counterexample found with bound 1
-- specification G ((state = slow & brake) -> X !(state = stop)) is false
-- as demonstrated by the following execution sequence
Trace Description: BMC Counterexample
Trace Type: Counterexample
-- Loop starts here
-> State: 4.1 <-
    state = stop
-> Input: 4.2 <-
    accelerate = TRUE

```



```

    brake = FALSE
-> State: 4.2 <-
    state = slow
-> Input: 4.3 <-
    accelerate = FALSE
    brake = TRUE
-> State: 4.3 <-
    state = stop
-- no counterexample found with bound 0
-- no counterexample found with bound 1
-- no counterexample found with bound 2
-- specification G ((state = fast & (!accelerate & !brake)) -> X !(state = slow)) is
false
-- as demonstrated by the following execution sequence
Trace Description: BMC Counterexample
Trace Type: Counterexample
-> State: 5.1 <-
    state = stop
-> Input: 5.2 <-
    accelerate = TRUE
    brake = FALSE
-- Loop starts here
-> State: 5.2 <-
    state = slow
-> Input: 5.3 <-
-> State: 5.3 <-
    state = fast
-> Input: 5.4 <-
    accelerate = FALSE
-> State: 5.4 <-
    state = slow
-- no counterexample found with bound 0
-- no counterexample found with bound 1
-- no counterexample found with bound 2
-- specification G ((state = fast & brake) -> X !(state = stop)) is false

```

-- as demonstrated by the following execution sequence

Trace Description: BMC Counterexample

Trace Type: Counterexample

-- Loop starts here

-> State: 6.1 <-

state = stop

-> Input: 6.2 <-

accelerate = TRUE

brake = FALSE

-> State: 6.2 <-

state = slow

-> Input: 6.3 <-

-> State: 6.3 <-

state = fast

-> Input: 6.4 <-

accelerate = FALSE

brake = TRUE

-> State: 6.4 <-

state = stop

The five additional trap properties and test cases are :

**1) G ((state = slow & (accelerate & !brake)) -> X !(state = fast))**

|         | t=0                 | t=1                 | t=2  | t=3 |
|---------|---------------------|---------------------|------|-----|
| Inputs  | Accelerate & !Brake | Accelerate & !Brake | \    |     |
| Outputs | stop                | slow                | fast |     |

**2) G ((state = slow & (!accelerate & !brake)) -> X !(state = stop))**

|         | t=0                 | t=1                  | t=2  | t=3 |
|---------|---------------------|----------------------|------|-----|
| Inputs  | Accelerate & !Brake | !Accelerate & !Brake | \    |     |
| Outputs | stop                | slow                 | stop |     |

3)  $G ((state = slow \ \& \ brake) \rightarrow X \ !(state = stop))$

|         | t=0                 | t=1                 | t=2  | t=3 |
|---------|---------------------|---------------------|------|-----|
| Inputs  | Accelerate & !Brake | !Accelerate & Brake | \    |     |
| Outputs | stop                | slow                | stop |     |

4)  $G ((state = fast \ \& \ (!accelerate \ \& \ !brake)) \rightarrow X \ !(state = slow))$

|         | t=0                 | t=1                 | t=2                  | t=3  |
|---------|---------------------|---------------------|----------------------|------|
| Inputs  | Accelerate & !Brake | Accelerate & !Brake | !Accelerate & !Brake | \    |
| Outputs | stop                | slow                | fast                 | slow |

5)  $G ((state = fast \ \& \ brake) \rightarrow X \ !(state = stop))$

|         | t=0                 | t=1                 | t=2                 | t=3  |
|---------|---------------------|---------------------|---------------------|------|
| Inputs  | Accelerate & !Brake | Accelerate & !Brake | !Accelerate & Brake | \    |
| Outputs | stop                | slow                | fast                | stop |